

PROGETTO FINALE: “Support Agent con RAG” (Python + OpenAI)

Obiettivo del progetto

Realizzare un mini sistema di Customer Care che:

1. riceve un messaggio cliente (da terminale)
2. lo classifica (tipo richiesta + urgenza)
3. recupera info da una knowledge base locale (RAG)
4. genera una bozza di risposta **usando solo il contesto**
5. salva un report in `runs/` (JSON + TXT)

Struttura cartelle e file da creare

Creare questa struttura:

```
src/  
  main.py  
  agent.py  
  api_client.py  
  prompts.py  
  classifier.py  
  retrieval.py  
  draft_reply.py  
  reporting.py  
  kb/  
    knowledge.json  
  runs/    (creata automaticamente)
```

1) Knowledge Base (kb/knowledge.json)

Creare kb/knowledge.json con una lista di oggetti:

- `id`: stringa breve
- `text`: testo informativo

Esempio (minimo 5 voci, meglio 30):

```
[  
  {"id": "checkin", "text": "Check-in dalle 14:00..."},  
  {"id": "colazione", "text": "Colazione 07:30-10:00..."}  
]
```

 È qui che il RAG prende le informazioni.

Se un'informazione non è qui dentro → l'agente non deve inventarla.

2) api_client.py — Wrapper OpenAI

Scopo

Centralizzare le chiamate OpenAI per non ripetere codice in più file.

Funzioni richieste

```
chat(messages, model="gpt-4o-mini", temperature=0.2)  
-> str
```

- Input: lista di messaggi in formato OpenAI:

```
[{"role": "user", "content": "..."}]
```

- Output: testo risposta (stringa)

`embed(texts, model="text-embedding-3-small") -> list[list[float]]`

- Input: lista di stringhe
- Output: lista di vettori embedding

🔑 Deve leggere la chiave da `key.env`.

3) prompts.py — Prompt templates

Scopo

Mettere in un unico file tutti i prompt.

Funzioni richieste

`build_classifier_prompt(message: str) -> str`

Deve chiedere all'AI di rispondere **solo JSON valido** con:

- label: INFO | RIMBORSO | CHECKIN | COLAZIONE | ASSISTENZA | ALTRO
- urgency: BASSA | MEDIA | ALTA
- summary: stringa breve

`build_reply_prompt(user_message, label, urgency, summary, context) -> str`

Deve imporre guardrail:

- usare **solo** il contesto

- se non basta → scrivere ESATTAMENTE:

Non ho informazioni sufficienti.

4) classifier.py — Classificazione messaggio

Scopo

Capire la tipologia della richiesta e l'urgenza.

Funzione richiesta

classify_message(message: str) -> dict

Ritorna:

```
{  
    "label": "...",  
    "urgency": "...",  
    "summary": "..."  
}
```

✅ Deve:

- chiamare OpenAI usando `api_client.chat()`
- usare il prompt da `prompts.build_classifier_prompt()`
- leggere JSON e validarlo

✅ Bonus (consigliato):

- se l'AI non restituisce JSON valido → fallback con regole (keyword)

5) retrieval.py — RAG (Recupero contesto)

Scopo

Recuperare i documenti più rilevanti dalla knowledge base e costruire il contesto.

Funzioni richieste

load_kb() -> list[dict]

- legge kb/knowledge.json
- ritorna la lista di documenti

retrieve_context(query: str, top_k: int = 3) -> dict

Ritorna:

```
{
  "context": "stringa con docs top",
  "top_k": ["id1", "id2", "id3"],
  "scores": [0.12, 0.09, 0.07]
}
```

 Deve:

- calcolare embedding docs
- calcolare embedding query
- calcolare similarity con dot product
- scegliere i migliori top_k
- costruire contesto tipo:

[checkin] ...

- [colazione] ...

📌 Qui entra il RAG.

6) draft_reply.py — Bozza risposta

Scopo

Generare una risposta “umana”, ma controllata.

Funzione richiesta

draft_reply(user_message, label, urgency, summary, context) -> str

✅ Deve:

- costruire prompt con guardrail da `prompts.build_reply_prompt()`
- chiamare `api_client.chat()`
- restituire la risposta

7) reporting.py — Salvataggio output

Scopo

Salvare tutto per debug e verifiche.

Funzione richiesta

save_run_report(..., out_dir="runs") -> dict

Deve creare:

- `runs/run_TIMESTAMP.json`
- `runs/reply_TIMESTAMP.txt`

Il JSON deve contenere:

- user_message
- classification
- context
- reply
- meta (top_k + scores)

Output funzione:

```
{  
  "report_json": "...",  
  "reply_txt": "..."  
}
```

8) agent.py — Orchestratore

Scopo

Mettere insieme tutto.

Classe richiesta

SupportAgent(top_k=3)

Metodo:

run(user_message: str) -> AgentResult

- chiama:
 - classifier
 - retrieval
 - draft_reply
 - reporting

Ritorna:

- label, urgency, summary
- context
- reply
- meta (file salvati)

9) main.py — Programma eseguibile

Scopo

Far funzionare tutto da terminale.



Deve:

1. leggere messaggio utente (anche multi-linea)
2. chiamare l'agente
3. stampare risposta finale
4. stampare paths dei file generati